

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Scala for D.S Practical

Practical - 21,22,23,24

Roll No. :S039	Name: Divyanshu singh
Class :SY-Bsc.IT	Batch : 00
Date of Assignment :-25-08-2026	Date/Time of Submission : 25-08-2026

PRACTICAL 21

Aim:

Join two CSV files in Spark DataFrames based on a common column and write the output to a file.

Code:

```
ThisBuild / version := "0.1.0-SNAPSHOT"

ThisBuild / scalaVersion := "2.12.18"
libraryDependencies += Seq(
  "org.apache.spark" %% "spark-core" % "3.5.1",
  "org.apache.spark" %% "spark-sql" % "3.5.1"
)
lazy val root = (project in file("."))
  .settings(
    name := "untitled"
  )
```

```
import org.apache.spark.sql.Session

object Practical21 extends App {
  val s = SparkSession.builder()
    .appName("JoinTwoCSV")
    .master("local[*]")
    .getOrCreate()

  val d1 = s.read
    .option("header", "true")
    .csv("File1.csv")

  val d2 = s.read
    .option("header", "true")
    .csv("File2.csv")

  val j = d1.join(d2, Seq("id"), "inner")

  j.show()

  j.write.option("header", "true")

  s.stop()
}
```

Output:

```
+---+-----+-----+
| id|  name|score|
+---+-----+-----+
|  1|  Rani|   85|
|  2|Sachin|   40|
|  3| Priya|   78|
+---+-----+-----+
```

PRACTICAL 22

Aim:

Set up Apache Spark locally and count the frequency of words in a text file.

Code:

```
import org.apache.spark.sql.SparkSession

object Practical22 extends App { new *
  val s = SparkSession.builder()
    .appName("WordCountExample")
    .master("local[*]")
    .getOrCreate()

  val r = s.sparkContext.textFile("input.txt")
  val w = r.flatMap(line => line.split("\\s+"))
  val wo = w.map(word => (word, 1))
  val wc = wo.reduceByKey(_ + _)
  wc.collect().foreach(println)
  s.stop()
}
```

Output:

```
26/08/25 10:54:01 INF
(is,2)
(easy,1)
(powerful,1)
(Spark,2)
26/08/25 10:54:01 INF
```

PRACTICAL 23

AIM:

Filter rows in a CSV file using Spark DataFrames where a numeric column exceeds a threshold.

Code:

```
build.sbt  Prac23.scala x superstore.csv

import org.apache.spark.sql.SparkSession

object Prac23 extends App {

  val spark = SparkSession.builder()
    .appName("FilterCSVExample")
    .master("local[*]")
    .getOrCreate()

  import spark.implicits._

  val df = spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("src/main/resources/superstore.csv")

  val f = df.filter($"Sales" > 5000)

  println("Filtered Data:")
  f.show()

  spark.stop()
}
```

Output:

```
20/08/25 10:54:47 INFO DAGScheduler: Job 2 is running. Submitting prioritized dependencies to cluster manager. Job 2 is
20/08/25 10:54:47 INFO TaskSchedulerImpl: Killing all running tasks in stage 2: Stage finished
20/08/25 10:54:47 INFO DAGScheduler: Job 2 finished: show at Prac23.scala:21, took 0.284635 s
20/08/25 10:54:47 INFO CodeGenerator: Code generated in 14.4766 ms
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID
1	1066	CA-2014-139892	9/8/2014	9/12/2014	Standard Class	BR-11140	Becky Martin	Consumer	United States	San Antonio	Texas	78207	Central TEC-MA-10000823
1	5101	CA-2015-145352	3/16/2015	3/22/2015	Standard Class	CM-12385	Christopher Martinez	Consumer	United States	Atlanta	Georgia	30318	South OFF-BI-10003527
1	684	US-2017-108116	11/4/2017	11/4/2017	Same Day	GT-14635	Grant Thornton	Corporate	United States	Burlington	North Carolina	27217	South TEC-MA-10004125
1	2586	CA-2014-143917	7/25/2014	7/27/2014	Second Class	KL-16045	Ken Lonsdale	Consumer	United States	San Francisco	California	94122	West OFF-SU-10000151
1	2624	CA-2017-127180	10/22/2017	10/24/2017	First Class	TA-21385	Tom Ashbrook	Home Office	United States	New York City	New York	10024	East TEC-CO-10004722
1	2698	CA-2014-145317	3/18/2014	3/23/2014	Standard Class	SH-20320	Sean Miller	Home Office	United States	Jacksonville	Florida	32216	South TEC-MA-10002412
1	4099	CA-2014-116904	9/23/2014	9/28/2014	Standard Class	SC-20095	Sanjit Chand	Consumer	United States	Minneapolis	Minnesota	55407	Central OFF-BI-10001128
1	4191	CA-2017-166709	11/17/2017	11/22/2017	Standard Class	HL-15040	Hunter Lopez	Consumer	United States	Newark	Delaware	19711	East TEC-CO-10004722
1	4278	US-2016-107448	4/16/2016	4/28/2016	Standard Class	BS-11365	Bill Shonely	Corporate	United States	Lakewood	New Jersey	8701	East TEC-MA-10001847
1	6341	CA-2017-143112	10/5/2017	10/9/2017	Standard Class	TS-21370	Todd Sumrall	Corporate	United States	New York City	New York	10035	East TEC-MA-10001847
1	6426	CA-2016-143714	5/23/2016	5/27/2016	Standard Class	CC-12370	Christopher Conant	Consumer	United States	Philadelphia	Pennsylvania	19120	East TEC-CO-10004722
1	6521	CA-2017-138289	1/16/2017	1/18/2017	Second Class	AR-10540	Andy Reiter	Consumer	United States	Jackson	Michigan	49201	Central OFF-BI-10004993
1	6627	CA-2014-145541	12/14/2014	12/21/2014	Standard Class	TB-21460	Tom Boeckenhauer	Consumer	United States	New York City	New York	10024	East TEC-MA-10001127
1	6827	CA-2016-118609	10/2/2016	10/9/2016	Standard Class	TC-20980	Tamara Chand	Corporate	United States	Lafayette	Indiana	47905	Central TEC-CO-10004722
1	7667	US-2016-140158	10/4/2016	10/8/2016	Standard Class	DR-12940	Daniel Raglin	Home Office	United States	Providence	Rhode Island	2988	East TEC-CO-10001449
1	8154	CA-2017-140151	3/23/2017	3/25/2017	First Class	RB-19360	Raymond Buch	Consumer	United States	Seattle	Washington	98115	West TEC-CO-10004722
1	8489	CA-2016-158842	2/2/2016	2/4/2016	Second Class	SE-20110	Sanjit Engle	Consumer	United States	Arlington	Virginia	22204	South TEC-MA-10001127
1	8859	CA-2017-135909	10/13/2017	10/26/2017	Standard Class	JW-15220	Jane Waco	Corporate	United States	Sacramento	California	95823	West OFF-BI-10003527
1	9040	CA-2016-117121	12/17/2016	12/21/2016	Standard Class	AB-10165	Adrian Barton	Consumer	United States	Detroit	Michigan	48205	Central OFF-BI-10000543

1	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Pct	680	^	v	:
2	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	C6-12520	Claire Gute	Consumer	United States	Henderson	Kentucky					
3	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	C6-12520	Claire Gute	Consumer	United States	Henderson	Kentucky					
4	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	Cal					
5	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	S0-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale						
6	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	S0-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale						
7	6	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Cal					
8	7	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Cal					
9	8	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Cal					
10	9	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Cal					
11	10	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Cal					
12	11	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Cal					
13	12	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	Cal					
14	13	CA-2017-114412	4/15/2017	4/20/2017	Standard Class	AA-10480	Andrew Allen	Consumer	United States	Concord	North Ca					
15	14	CA-2016-161389	12/5/2016	12/10/2016	Standard Class	IM-15070	Irene Maddox	Consumer	United States	Seattle	Washing					
16	15	US-2015-118983	11/22/2015	11/26/2015	Standard Class	HP-14815	Harold Pawlan	Home Office	United States	Fort Worth						
17	16	US-2015-118983	11/22/2015	11/26/2015	Standard Class	HP-14815	Harold Pawlan	Home Office	United States	Fort Worth						
18	17	CA-2014-105893	11/11/2014	11/18/2014	Standard Class	PK-19075	Pete Kriz	Consumer	United States	Madison	Wisconsin					
19	18	CA-2014-167164	5/13/2014	5/15/2014	Second Class	A6-10270	Alejandro Grove	Consumer	United States	West Jordan	Utah					
20	19	CA-2014-143336	8/27/2014	9/1/2014	Second Class	ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco	Calif					
21	20	CA-2014-143336	8/27/2014	9/1/2014	Second Class	ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco	Calif					
22	21	CA-2014-143336	8/27/2014	9/1/2014	Second Class	ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco	Calif					
23	22	CA-2016-137330	12/9/2016	12/13/2016	Standard Class	KB-16585	Ken Black	Corporate	United States	Fremont	Nebraska					
24	23	CA-2016-137330	12/9/2016	12/13/2016	Standard Class	KB-16585	Ken Black	Corporate	United States	Fremont	Nebraska					
25	24	US-2017-156909	7/16/2017	7/18/2017	Second Class	SF-20065	Sandra Flanagan	Consumer	United States	Philadelphia	Pa					
26	25	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	Emily Burns	Consumer	United States	Orem	Utah	84057				
27	26	CA-2016-121755	1/16/2016	1/20/2016	Second Class	EH-13945	Eric Hoffmann	Consumer	United States	Los Angeles	Calif					
28	27	CA-2016-121755	1/16/2016	1/20/2016	Second Class	EH-13945	Eric Hoffmann	Consumer	United States	Los Angeles	Calif					
29	28	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	Tracy Blumstein	Consumer	United States	Philadelphia						
30	29	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	Tracy Blumstein	Consumer	United States	Philadelphia						

PRACTICAL 24

AIM:

Perform a group-by operation in Spark DataFrames to compute the average of each group.

Code:

```
Current File ▾
build.sbt  Prac24.scala x  superstore.csv
1
2 import org.apache.spark.sql.SparkSession
3 import org.apache.spark.sql.functions._
4
5 object Prac24 extends App {
6
7     val spark = SparkSession.builder()
8         .appName("AverageSales")
9         .master("local[*]")
10        .getOrCreate()
11
12    val df = spark.read
13        .option("header", "true")
14        .option("inferSchema", "true")
15        .csv("src/main/resources/superstore.csv")
16
17    val result = df.groupBy("Category")
18        .agg(avg("Sales").alias("Average Sales"))
19
20    println("Average Sales by Category:")
21    result.show()
22
23    spark.stop()
24 }
```

Prac24

Output:

```
26/08/25 11:12:42 INFO SparkUI: Stopped Spark
+-----+-----+
| Category| Average Sales|
+-----+-----+
|Office Supplies|121.69225531914947|
| Furniture| 353.4459311957568|
| Technology| 454.5405475802047|
+-----+-----+
```